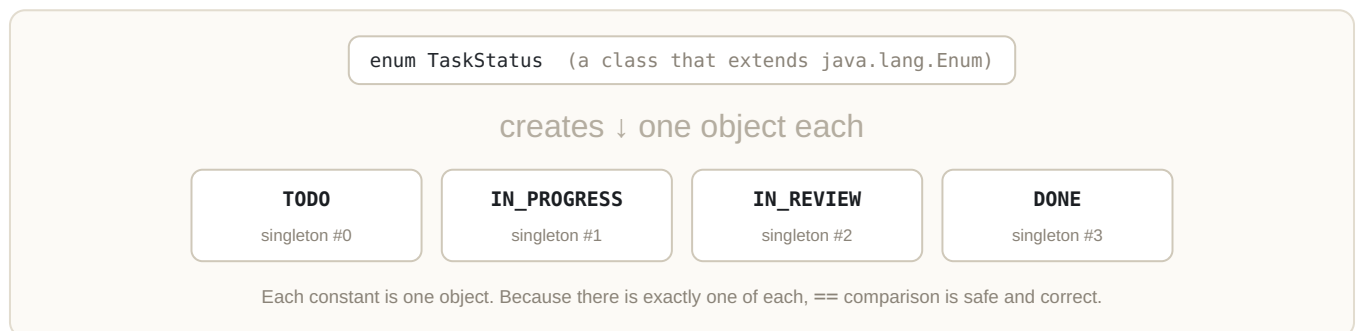


Java Enums

Comprehensive notes & interview preparation — distilled from building a task state machine (`TaskStatus.canTransitionTo`) in a Spring Boot workspace service.

1 · The core idea: an enum is a class

The single biggest shift from "enums are just named constants" is this: in Java an **enum is a full class**, and each constant you list is a **singleton object** of that class, created for you by the JVM. You never write `new` — Java instantiates exactly one object per constant, once.



Consequences that fall out of "it's a class":

- An enum **implicitly extends** `java.lang.Enum`, so it cannot extend another class (Java has single inheritance). It *can* implement interfaces.
- An enum is **implicitly final** (unless it has constant-specific bodies) — you cannot subclass it.
- The **constructor is implicitly private**. You cannot create new instances at runtime, which is what makes the set "fixed."
- It is **type-safe**: a method taking `TaskStatus` can only receive one of the four constants — not an arbitrary `int` or `String`.

2 · Enums can hold data and behavior

Because it's a class, an enum can declare fields, a constructor, and methods. This is the leap from "storing constants" to "modelling a concept." A constant can carry data:

```
public enum TaskPriority {
    LOW(1), MEDIUM(2), HIGH(3), CRITICAL(4);    // args go to the private constructor

    private final int weight;
    TaskPriority(int weight) { this.weight = weight; }    // implicitly private
    public int getWeight() { return weight; }
}
```

...and, more powerfully, it can carry **behavior** — a method that answers a question about the constant it is called on. That is exactly what the state-machine rulebook does (Section 4).

2.1 · What this means inside an enum method

It means the same thing it does in any class: **the object the method was called on**. If your code runs

`task.getTaskStatus().canTransitionTo(target)` and the status is `IN_PROGRESS`, then inside the method `this` is the `IN_PROGRESS` object.

```
IN_PROGRESS.canTransitionTo( IN_REVIEW )
```

↑ the object before the dot becomes **this** (the from-state) · ↑ the argument becomes **target** (the to-state)

`switch (this)` jumps to the `IN_PROGRESS` case, which checks `target == IN_REVIEW` → `true`.

2.2 · Built-in members every enum gets for free

Member	Returns	Notes
<code>values()</code>	array of all constants	In declaration order. A static method.
<code>valueOf("DONE")</code>	the matching constant	Case-sensitive; throws <code>IllegalArgumentException</code> if no match.
<code>name()</code>	the exact identifier, e.g. "DONE"	<code>final</code> — cannot be overridden.
<code>ordinal()</code>	0-based position	Avoid relying on it for logic or storage (see §5.1).
<code>toString()</code>	defaults to <code>name()</code>	Can be overridden for display text.
<code>compareTo()</code>	order by ordinal	Enums implement <code>Comparable</code> .

3 · Control flow: switch, equality, null

3.1 · switch on an enum

Inside an enum `switch`, you write the constant **unqualified** — `case TODO`, not `case TaskStatus.TODO` — because the compiler already knows the type. Two syntaxes:

Traditional (colon) — execution "falls through" to the next case unless stopped by `break` or `return`.

```
switch (this) {
    case TODO:
        return target == IN_PROGRESS;
    case IN_PROGRESS:
        return target == IN_REVIEW
            || target == TODO;
    default:
        return false;
}
```

Arrow (Java 14+) — each branch is self-contained; no fall-through, no `break` needed.

```
return switch (this) {
    case TODO      -> target == IN_PROGRESS;
    case IN_PROGRESS -> target == IN_REVIEW
                    || target == TODO;
    default        -> false;
};
```

Exhaustiveness

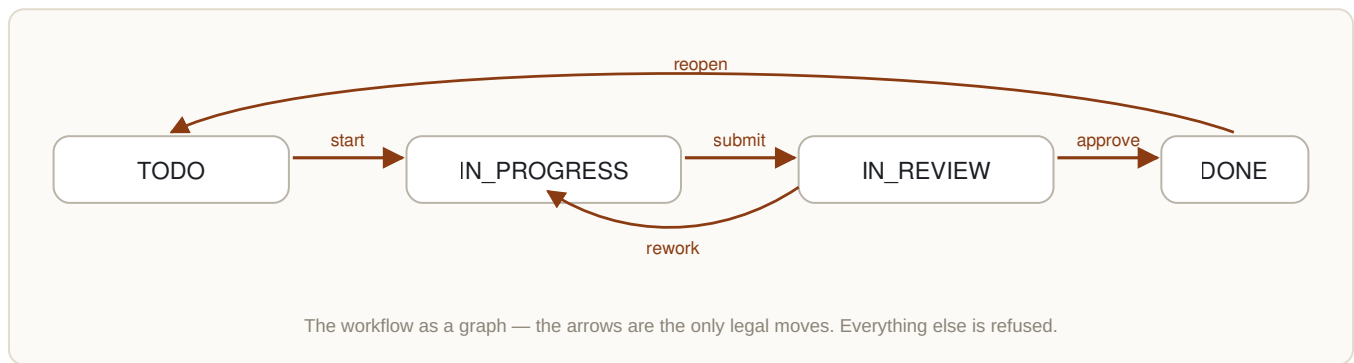
Covering every constant makes the switch exhaustive. Adding a default `-> false` is a cheap safety net: if someone adds a new constant later and forgets to give it rules, the code still compiles and behaves predictably (deny by default).

3.2 · Comparing enums: `==` vs `equals()`

Both work, but `==` is preferred for enums. Because each constant is a singleton, identity comparison is also value comparison. Crucially, `==` is **null-safe**: `status == DONE` never throws even if `status` is null (it's just `false`), whereas `status.equals(DONE)` would throw a `NullPointerException`. This is why `target == IN_PROGRESS` inside the rulebook needs no null check.

4 · Smart enums: encoding rules (the case study)

The most valuable pattern: put the **rules about the constants on the enum itself**, so there is one authoritative place that owns them. Our task state machine is exactly this — a method that answers "from my current state, is moving to `target` allowed?"



The same arrows as a lookup — this is precisely what `canTransitionTo` returns for every pair:

from ↓ to →	TODO	IN_PROGRESS	IN_REVIEW	DONE
TODO	—	✓ start		
IN_PROGRESS	✓ back	—	✓ submit	
IN_REVIEW		✓ rework	—	✓ approve
DONE	✓ reopen			—

4.1 · Why put the rule on the enum?

- **Single source of truth.** The allowed moves live in one method. The service only *asks*; it never re-decides. Change the rules once, everywhere stays consistent.
- **Cohesion.** Knowledge about statuses lives with the status type, not scattered as `if (status == X && target == Y)` across service classes.
- **Testability.** The rulebook is a pure function — no database, no Spring — so it is trivial to unit-test in isolation.

4.2 · The static-initializer gotcha (and why switch avoids it)

Trap: building an EnumMap of transitions in a static block

If you try to declare `static final Map<TaskStatus, Set<TaskStatus>> ALLOWED` and populate it in a static `{ }` block that references the constants, you can touch a constant before it has finished being constructed, producing a confusing `NullPointerException` at class-load time. A plain `switch` (this) sidesteps this entirely — it needs no static state — and it reads like the matrix above.

4.3 · Constant-specific behavior & interfaces (advanced)

An enum can declare an **abstract method** that each constant implements differently — polymorphism with no `if / switch` at all:

```

public enum Operation {
    PLUS { public int apply(int a, int b) { return a + b; } },
    MINUS { public int apply(int a, int b) { return a - b; } };
    public abstract int apply(int a, int b);
}

```

Enums can also **implement interfaces**, letting different enum types share a contract (e.g. a common `Labelled` interface with `getLabel()`).

5 · Enums in a Spring Boot application

5.1 · JPA persistence — @Enumerated(STRING) vs ORDINAL

When an enum field is stored by JPA/Hibernate, you choose *how* it is written to the column. This is a decision with real data-integrity consequences.

`@Enumerated(EnumType.STRING)` ✓

```
status = 'IN_REVIEW'
```

Stores the **name**. Readable, and safe to reorder or insert constants.

`@Enumerated(EnumType.ORDINAL)` ✗

```
status = 2
```

Stores the **position**. Insert **BLOCKED** at index 2 later and every old **2** now means the wrong state.

ORDINAL couples your stored data to declaration order — a silent corruption risk. Prefer STRING.

Rule of thumb

Always annotate persisted enum fields with `@Enumerated(EnumType.STRING)`. The default is `ORDINAL`, so this is an *active* choice you must make. Never build logic on `ordinal()` either.

5.2 · JSON (de)serialization with Jackson

- By default Jackson maps a JSON string to the enum constant **by exact name** (`"IN_REVIEW"` → `IN_REVIEW`), case-sensitively.
- An **invalid value** (e.g. `"BANANA"`) fails deserialization → Spring raises `HttpMessageNotReadableException` , normally surfaced as **400 Bad Request**. (Beware a broad `@ExceptionHandler(RuntimeException.class)` catch-all, which can turn this into a 500.)
- To accept custom wire values (lowercase, aliases), use `@JsonValue` (serialize) and `@JsonCreator` (deserialize) on the enum.

5.3 · Enums as request-DTO fields

An enum field in a request DTO gives you **free type validation** at the boundary: the client literally cannot send a value outside the fixed set (a bad one is rejected during parsing). Pair with Bean Validation (`@NotNull`) to also reject a missing value cleanly.

6 · Collections & the singleton pattern

6.1 · EnumSet and EnumMap

For sets or maps keyed by an enum, prefer the specialized `EnumSet` and `EnumMap` . They are backed by a bit-vector / plain array indexed by `ordinal()` , so they are dramatically faster and lighter than `HashSet` / `HashMap` , and iterate in declaration order.

```
EnumSet<TaskStatus> openStates = EnumSet.of(TODO, IN_PROGRESS, IN_REVIEW);
boolean isOpen = openStates.contains(status);
```

6.2 · Enums as singletons

A single-constant enum is the most robust **Singleton** in Java (Effective Java, Item 3): the JVM guarantees one instance, it is thread-safe on creation, and it is serialization-safe — deserialization returns the existing constant (via built-in `readResolve`) rather than making a copy.

7 · Best-practice checklist

Do	Why
Persist with <code>@Enumerated(EnumType.STRING)</code>	Decouples stored data from declaration order.
Compare with <code>==</code>	Correct for singletons and null-safe.
Put rules/behavior on the enum (smart enum)	Single source of truth; high cohesion.
Use <code>EnumSet</code> / <code>EnumMap</code> for enum-keyed collections	Faster, clearer, ordered.
Add a <code>default</code> branch in switches	Safe behavior when a new constant appears.
Validate enum inputs at the boundary	Clean 400s instead of deep failures.
Never build logic on <code>ordinal()</code>	It shifts the moment you reorder constants.

8 · Interview questions & answers

Grouped from fundamentals to advanced. Answers are kept tight enough to say out loud.

Fundamentals

Q1. What is an enum in Java, really?

A special kind of class representing a fixed set of constants. Each constant is a **singleton instance** of the enum type, created by the JVM. It implicitly extends `java.lang.Enum`, is implicitly `final`, and is type-safe.

Q2. Can an enum have fields, constructors, and methods?

Yes. It's a class, so it can attach data (via a constructor) and behavior (methods) to its constants. The constructor is implicitly **private** — you cannot create new instances.

Q3. Why can't you use `new` on an enum?

The constructor is private and the constant set is fixed at compile time. That fixedness is the whole guarantee an enum provides.

Q4. Difference between `name()`, `toString()`, and `ordinal()`?

`name()` returns the exact identifier and is `final`. `toString()` defaults to `name()` but can be overridden for display. `ordinal()` is the 0-based declaration position — useful to the language, dangerous to depend on.

Q5. What does `values()` do, and `valueOf(String)`?

`values()` returns an array of all constants in order. `valueOf("DONE")` returns the matching constant or throws `IllegalArgumentException`; it is case-sensitive.

Comparison & control flow

Q6. `==` or `equals()` for enums?

Prefer `==`: each constant is a singleton, so identity equals value, and `==` is null-safe (`equals()` can NPE on a null left side).

Q7. In a `switch` on an enum, why do you write `case TODO` and not `case TaskStatus.TODO`?

The switch already knows the type from the value being switched on, so the label must be the unqualified constant name; qualifying it is a compile error.

Q8. Traditional vs arrow switch?

Traditional (`case X:`) falls through to the next case unless you `break / return`. Arrow (`case X ->`, Java 14+) is self-contained with no fall-through. Both are fine; arrow is safer by default.

Q9. Does comparing a null enum with `==` throw?

No. `null == DONE` is simply `false`. That's why the state-machine rulebook needs no explicit null guard.

Design & the smart-enum pattern

Q10. How would you model a state machine with an enum?

Give the enum a method like `canTransitionTo(target)` that returns whether the move is a legal edge — one authoritative rulebook. The service loads the entity, asks the rulebook, and either applies the change or rejects it (e.g. 409/422). The enum owns the rules; the service just consults them.

Q11. Why put the transition rules on the enum instead of in the service?

Single source of truth, high cohesion (the knowledge lives with the type), and easy pure-unit testing. Scattering `if` checks across services duplicates the rules and drifts.

Q12. What is a constant-specific method body?

An `abstract` method on the enum that each constant overrides, giving per-constant behavior without any `if / switch` — clean polymorphism (classic calculator `Operation` example).

Q13. Can an enum extend a class? Implement an interface?

It cannot extend a class (it already extends `java.lang.Enum`, and Java is single-inheritance). It **can** implement any number of interfaces.

Persistence, JSON & advanced

Q14. Why `@Enumerated(EnumType.STRING)` over `ORDINAL` in JPA?

STRING stores the constant's name — stable and readable. ORDINAL stores its position, so reordering or inserting a constant silently remaps existing rows to the wrong value. STRING must be set explicitly because ORDINAL is the default.

Q15. What happens when Jackson receives an invalid enum string?

Deserialization fails with `HttpMessageNotReadableException`, normally a 400. You can customize accepted values with `@JsonCreator` / `@JsonValue`. Watch out for a catch-all `RuntimeException` handler masking the 400 as a 500.

Q16. What are `EnumSet` and `EnumMap`, and when do you use them?

Enum-specialized collections backed by a bit-vector / array indexed by ordinal. Use them for any set/map keyed by an enum — faster and lighter than hash-based ones, and iterate in declaration order.

Q17. Why is an enum the best Singleton in Java?

The JVM guarantees exactly one instance, creation is thread-safe, and serialization returns the existing constant (built-in `readResolve`) instead of a copy — so the singleton survives serialization, unlike a hand-rolled one.

Q18. What's the gotcha with a static `EnumMap` of transitions built inside the enum?

A static initializer can reference constants before they are fully constructed, throwing an `NPE` at class-load. Avoid it with a `switch`-based method (no static state) or lazy initialization.

Q19. Are enum values still singletons after serialization/deserialization?

Yes. Enums serialize by name and deserialize back to the canonical constant, so identity (`==`) is preserved across the round trip.

Q20. When would you promote an enum to a database lookup table instead?

When users must define values at runtime, or each value needs rich metadata/relationships. A fixed, code-owned set stays an enum; an open, data-owned set becomes a table with a foreign key.

Generated as study notes for the Team Collaboration App (workspace-service). Code examples reference the project's `TaskStatus` state machine.